

九宫格按键输入

题目描述

九宫格 按键输入，有英文和数字两个模式，默认是数字模式，数字模式直接输出数字，英文模式连续按同一个按键会依次出现这个按键上的字母，如果输入 "/" 或者其他字符，则循环中断，输出此时停留的字母。

数字和字母的对应关系如下，注意 0 只对应空格：

1 .,	2 abc	3 def
4 ghi	5 jkl	6 mno
7 pqrs	8 tuv	9 wxyz
#	0 空格	/

输入一串按键，要求输出屏幕显示

1. # 用于切换模式，默认是数字模式，执行 # 后切换为英文模式；
2. / 表示延迟，例如在英文模式下，输入 22/222，显示为 bc，数字模式下 / 没有效果；
3. 英文模式下，多次按同一键，例如输入 22222，显示为 b；

输入描述

输入范围为数字 0~9 和字符 '#'、'/'，输出屏幕显示，例如：

- 在数字模式下，输入 1234，显示 1234
- 在英文模式下，输入 1234，显示 ,adg

输出描述

输出屏幕显示的字符

用例

输入	2222/22
输出	222222
说明	默认数字模式，字符直接显示，数字模式下/无序

输入	#2222/22
输出	ab
说明	#进入英文模式，连续的数字输入会循环选择字母，直至输入/，故第一段2222输入显示a，第二段22输入显示b

输入	#222233
输出	ae
说明	#进入英文模式，连续的数字输入会循环选择字母，直至输入其他数字，故第一段2222输入显示a，第二段33输入显示e

题目解析

本题主要考察逻辑模拟。

首先，我们可以创建一个长度为10的数组，数组索引与数字0~9对应，数组元素就是数字对应的字母集合（**字符串**）。具体可以看代码实现中dic数组。

然后，本题的按键逻辑如下：

我们可以定义两个变量：delay_num 记录英文模式下按键数字，delay_idx 记录英文模式下当前按键数字停留的字母索引位置。
比如 dic[delay_num][delay_idx] 就是英文模式下，当前按键停留的字母。

遍历输入串 s 的每一个字符 c：

- c == '#': 切换模式，并且有打断英文字母切换的功能
- c == '/': 打断英文字母切换的功能
- 其他情况：
 1. 如果当前是数字模式，则当前按键（数字）直接录入结果
 2. 如果当前是英文模式，则看当前按键和上一次按键是否相同：

若相同，则进行英文字母切换，即 $\text{delay_idx} = (\text{delay_idx} + 1) \% \text{dic}[\text{delay_num}].\text{length}$

若不相同，则打断英文字母切换的功能，上一次按键停留的英文字母 dic[delay_num][delay_idx] 录入结果，并更新当前按键为英文切换的按键：delay_num = c - '0', delay_idx = 0

按照上面逻辑，处理完输入串即可。

Java算法源码

```
1 | import java.util.Scanner;  
2 |  
3 | public class Main {
```

```

4 // 数字（数组索引）和字母（数组元素）的映射关系
5
static String[] dic = {" ", ",.", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"}; 6
7 // 当前是否为数字模式
8 static boolean isNumMode = true;
9
10 // 英文模式下，数字 delay_num 切换到了 delay_idx 字母，比如 delay_num = 2, delay_idx = 1, 表示字母 'a', 即: dic[delay_num][delay_idx]
11 static int delay_num = -1;
12 static int delay_idx = -1;
13
14 // 记录结果
15 static StringBuilder sb = new StringBuilder();
16
17 public static void main(String[] args) {
18     Scanner sc = new Scanner(System.in);
19
20     // 尾部追加一个 '/', 可以避免收尾操作，也不影响结果
21     String s = sc.nextLine() + "/";
22
23     // 遍历输入串
24     for (int i = 0; i < s.length(); i++) {
25         char c = s.charAt(i);
26
27         // 切换模式
28         if (c == '#') {
29             isNumMode = !isNumMode;
30         }
31
32         if (c == '#' || c == '/') {
33             // # 和 / 都会打断英文字母切换动作
34             interrupt(c);
35         } else if (isNumMode) {
36             // 数字模式下，数字直接录入结果
37             sb.append(c);
38         } else if (c - '0' == delay_num) {
39             // 英文模式下，如果当前按键c和上一次按键delay_num相同，则进行英文字母切换
40             delay_idx++;

```

```

41         delay_idx %= dic[delay_num].length(); // 避免越界
42     } else {
43         // 英文模式下，如果当前按键c和上一次按键delay_num不相同，则打断英文字母切换动作
44         interrupt(c);
45     }
46 }
47
48 System.out.println(sb);
49 }
50
51 // 按键c如果可以打断英文切换
52 public static void interrupt(char c) {
53     // 如果之前有英文输入
54     if (delay_num != -1) {
55         // 则循环中断，输出此时停留的字母
56         sb.append(dic[delay_num].charAt(delay_idx));
57     }
58
59     // 更新delay_num和delay_idx
60     if (c == '#' || c == '/') {
61         delay_num = -1;
62         delay_idx = -1;
63     } else {
64         delay_num = c - '0';
65         delay_idx = 0;
66     }
67 }
68 }

```

JS算法源码

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4

```

运行

```
5 // 数字（数组索引）和字母（数组元素）的映射关系 6 | const dic = [  
7   " ",  
8   ",.",  
9   "abc",  
10  "def",  
11  "ghi",  
12  "jkl",  
13  "mno",  
14  "pqrs",  
15  "tuv",  
16  "wxyz",  
17 ];  
18  
19 // 当前是否为数字模式  
20 let isNumMode = true;  
21  
22 // 英文模式下，数字 delay_num 切换到了 delay_idx 字母，比如 delay_num = 2, delay_idx = 1, 表示字母 'a'，即: dic[delay_num][delay_idx]  
23 let delay_num = -1;  
24 let delay_idx = -1;  
25  
26 // 记录结果  
27 const res = [];  
28  
29 void (async function () {  
30   // 尾部追加一个 '/', 可以避免收尾操作，也不影响结果  
31   const s = (await readline()) + "/";  
32  
33   // 遍历输入串  
34   for (let c of s) {  
35     // 切换模式  
36     if (c == "#") {  
37       isNumMode = !isNumMode;  
38     }  
39  
40     if (c == "#" || c == "/") {  
41       // # 和 / 都会打断英文字母切换动作  
42       interrupt(c);
```

```

43 |     } else if (isNumMode) {
44 |         // 数字模式下，数字直接录入结果
45 |         res.push(c);
46 |     } else if (parseInt(c) == delay_num) {
47 |         // 英文模式下，如果当前按键c和上一次按键delay_num相同，则进行英文字母切换
48 |         delay_idx++;
49 |         delay_idx %= dic[delay_num].length; // 避免越界
50 |     } else {
51 |         // 英文模式下，如果当前按键c和上一次按键delay_num不相同，则打断英文字母切换动作
52 |         interrupt(c);
53 |     }
54 | }
55 |
56 | console.log(res.join(""));
57 | })();
58 |
59 | // 按键c如果可以打断英文切换
60 | function interrupt(c) {
61 |     // 如果之前有英文输入
62 |     if (delay_num != -1) {
63 |         // 则循环中断，输出此时停留的字母
64 |         res.push(dic[delay_num][delay_idx]);
65 |     }
66 |
67 |     // 更新delay_num和delay_idx
68 |     if (c == "#" || c == "/") {
69 |         delay_num = -1;
70 |         delay_idx = -1;
71 |     } else {
72 |         delay_num = parseInt(c);
73 |         delay_idx = 0;
74 |     }
75 | }

```

Python算法源码

```
1 # 数字（数组索引）和字母（数组元素）的映射关系
2 dic = (" ", ",.", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz")
3
4 # 当前是否为数字模式
5 isNumMode = True
6
7 # 英文模式下，数字 delay_num 切换到了 delay_idx 字母，比如 delay_num = 2, delay_idx = 1, 表示字母 'a', 即: dic[delay_num][delay_idx]
8 delay_num = -1
9 delay_idx = -1
10
11 # 记录结果
12 res = []
13
14
15 # 按键c如果可以打断英文切换
16 def interrupt(c):
17     global delay_num
18     global delay_idx
19
20     # 如果之前有英文输入
21     if delay_num != -1:
22         # 则循环中断，输出此时停留的字母
23         res.append(dic[delay_num][delay_idx])
24
25     # 更新delay_num和delay_idx
26     if c == '#' or c == '/':
27         delay_num = -1
28         delay_idx = - 1
29     else:
30         delay_num = int(c)
31         delay_idx = 0
32
33
34 if __name__ == '__main__':
35     # 尾部追加一个 '/', 可以避免收尾操作，也不影响结果
```



```

36 | s = input() + '/'
    |                               37 |
38 | # 遍历输入串
39 | for c in s:
40 |     # 切换模式
41 |     if c == '#':
42 |         isNumMode = not isNumMode
43 |
44 |     if c == '#' or c == '/':
45 |         # # 和 / 都会打断英文字母切换动作
46 |         interrupt(c)
47 |     elif isNumMode:
48 |         # 数字模式下，数字直接录入结果
49 |         res.append(c)
50 |     elif int(c) == delay_num:
51 |         # 英文模式下，如果当前按键c和上一次按键delay_num相同，则进行英文字母切换
52 |         delay_idx += 1
53 |         delay_idx %= len(dic[delay_num])
54 |     else:
55 |         # 英文模式下，如果当前按键c和上一次按键delay_num不相同，则打断英文字母切换动作
56 |         interrupt(c)
57 |
58 | print("".join(res))

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <stdbool.h>
3 | #include <string.h>
4 |
5 | // 数字（数组索引）和字母（数组元素）的映射关系
6 | char *dic[] = {" ", ",.", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
7 |
8 | // 当前是否为数字模式

```

```

9 | bool isNumMode = true;
   |         10 |
11 | // 英文模式下, 数字 delay_num 切换到了 delay_idx 字母, 比如 delay_num = 2, delay_idx = 1, 表示字母 'a', 即: dic[delay_num][delay_idx]
12 | int delay_num = -1;
13 | int delay_idx = -1;
14 |
15 | // 记录结果
16 | char res[10000] = {'\0'};
17 |
18 | // 按键c如果可以打断英文切换
19 | void interrupt(char c) {
20 |     // 如果之前有英文输入
21 |     if (delay_num != -1) {
22 |         // 则循环中断, 输出此时停留的字母
23 |         res[strlen(res)] = dic[delay_num][delay_idx];
24 |     }
25 |
26 |     // 更新delay_num和delay_idx
27 |     if (c == '#' || c == '/') {
28 |         delay_num = -1;
29 |         delay_idx = -1;
30 |     } else {
31 |         delay_num = c - '0';
32 |         delay_idx = 0;
33 |     }
34 | }
35 |
36 | int main() {
37 |     char s[10000] = {'\0'};
38 |     scanf("%s", s);
39 |
40 |     // 尾部追加一个 '/', 可以避免收尾操作, 也不影响结果
41 |     s[strlen(s)] = '/';
42 |
43 |     // 遍历输入串
44 |     for (int i = 0; i < strlen(s); i++) {
45 |         char c = s[i];

```

```

46 |
47 |         // 切换模式
48 |         if (c == '#') {
49 |             isNumMode = !isNumMode;
50 |         }
51 |
52 |         if (c == '#' || c == '/') {
53 |             // # 和 / 都会打断英文字母切换动作
54 |             interrupt(c);
55 |         } else if (isNumMode) {
56 |             // 数字模式下，数字直接录入结果
57 |             res[strlen(res)] = c;
58 |         } else if (c - '0' == delay_num) {
59 |             // 英文模式下，如果当前按键c和上一次按键delay_num相同，则进行英文字母切换
60 |             delay_idx += 1;
61 |             delay_idx %= strlen(dic[delay_num]);
62 |         } else {
63 |             // 英文模式下，如果当前按键c和上一次按键delay_num不相同，则打断英文字母切换动作
64 |             interrupt(c);
65 |         }
66 |     }
67 |
68 |     printf("%s", res);
69 |
70 |     return 0;
71 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | // 数字（数组索引）和字母（数组元素）的映射关系
6 | vector<string> dic = {" ", ",", ".", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};

```

```
7 | 8 | // 当前是否为数字模式
9 | bool isNumMode = true;
10 |
11 | // 英文模式下, 数字 delay_num 切换到了 delay_idx 字母, 比如 delay_num = 2, delay_idx = 1, 表示字母 'a', 即: dic[delay_num][delay_idx]
12 | int delay_num = -1;
13 | int delay_idx = -1;
14 |
15 | // 记录结果
16 | string res;
17 |
18 | // 按键c如果可以打断英文切换
19 | void interrupt(char c) {
20 |     // 如果之前有英文输入
21 |     if (delay_num != -1) {
22 |         // 则循环中断, 输出此时停留的字母
23 |         res += dic[delay_num][delay_idx];
24 |     }
25 |
26 |     // 更新delay_num和delay_idx
27 |     if (c == '#' || c == '/') {
28 |         delay_num = -1;
29 |         delay_idx = -1;
30 |     } else {
31 |         delay_num = c - '0';
32 |         delay_idx = 0;
33 |     }
34 | }
35 |
36 | int main() {
37 |     string s;
38 |     cin >> s;
39 |
40 |     // 尾部追加一个 '/', 可以避免收尾操作, 也不影响结果
41 |     s += '/';
42 |
43 |     // 遍历输入串
44 |     for (const auto &c: s) {
```

```
45 | // 切换模式
    |         46 |         if (c == '#') {
47 |             isNumMode = !isNumMode;
48 |         }
49 |
50 |         if (c == '#' || c == '/') {
51 |             // # 和 / 都会打断英文字母切换动作
52 |             interrupt(c);
53 |         } else if (isNumMode) {
54 |             // 数字模式下，数字直接录入结果
55 |             res += c;
56 |         } else if (c - '0' == delay_num) {
57 |             // 英文模式下，如果当前按键c和上一次按键delay_num相同，则进行英文字母切换
58 |             delay_idx += 1;
59 |             delay_idx %= dic[delay_num].length();
60 |         } else {
61 |             // 英文模式下，如果当前按键c和上一次按键delay_num不相同，则打断英文字母切换动作
62 |             interrupt(c);
63 |         }
64 |     }
65 |
66 |     cout << res << endl;
67 |
68 |     return 0;
69 | }
```